HDOC Day-76 (14-Nov-26)

Enumeration in C

1. Enumeration for Months and Number of Days

Problem Statement

Create an enumeration for months and print how many days each month has. For input FEB, the output should be 28 or 29 days.

Algorithm / Logic

1. Define an enumeration containing JAN to DEC.
2. Read the month name from the user.
3. Convert the entered month into the corresponding enumeration value.
4. Use a switch statement to print 31 days, 30 days, or 28 or 29 days for February.
5. Print Invalid Month if the input does not match any month.

C Program

```
#include <stdio.h>
#include <string.h>

enum Month { JAN, FEB, MAR, APR, MAY, JUN,
             JUL, AUG, SEP, OCT, NOV, DEC };

int main() {
    char input[10];
    enum Month month;

    printf("Enter month (JAN, FEB, MAR, ...): ");
    scanf("%9s", input);

    if (strcmp(input, "JAN") == 0) month = JAN;
    else if (strcmp(input, "FEB") == 0) month = FEB;
    else if (strcmp(input, "MAR") == 0) month = MAR;
    else if (strcmp(input, "APR") == 0) month = APR;
    else if (strcmp(input, "MAY") == 0) month = MAY;
    else if (strcmp(input, "JUN") == 0) month = JUN;
    else if (strcmp(input, "JUL") == 0) month = JUL;
    else if (strcmp(input, "AUG") == 0) month = AUG;
    else if (strcmp(input, "SEP") == 0) month = SEP;
    else if (strcmp(input, "OCT") == 0) month = OCT;
    else if (strcmp(input, "NOV") == 0) month = NOV;
    else if (strcmp(input, "DEC") == 0) month = DEC;
    else {
        printf("Invalid Month\n");
        return 0;
    }

    switch (month) {
        case JAN: case MAR: case MAY: case JUL:
        case AUG: case OCT: case DEC:
            printf("31 days\n");
            break;
```

```
        case APR: case JUN: case SEP: case NOV:
            printf("30 days\n");
            break;
        case FEB:
            printf("28 or 29 days\n");
            break;
    }
    return 0;
}
```

Explanation

The enumeration Month assigns symbolic names to all twelve months. The entered month is matched using strcmp() and stored as an enum value. A switch statement then groups months according to their number of days. February is displayed as having 28 or 29 days because the exact value depends on whether the year is a leap year.

Test Cases

Test Case	Input	Expected Output
1	FEB	28 or 29 days
2	APR	30 days
3	DEC	31 days

Sample Output

```
Enter month (JAN, FEB, MAR, ...): FEB
28 or 29 days
```

Result

Thus, the C program using enumeration successfully identifies a month and prints the corresponding number of days.

2. Enumeration for Operation Status

Problem Statement

Define an enumeration with SUCCESS, FAILURE, and TIMEOUT, and print messages accordingly. For input FAILURE, the output should be Operation Failed.

Algorithm / Logic

1. Define an enumeration Status with SUCCESS, FAILURE, and TIMEOUT.
2. Read the status name from the user.
3. Match the entered text with the corresponding enumeration value.
4. Use a switch statement to print the appropriate operation message.
5. Print Invalid Status if the entered value is not recognized.

C Program

```
#include <stdio.h>
#include <string.h>

enum Status { SUCCESS, FAILURE, TIMEOUT };

int main() {
    char input[20];
    enum Status status;

    printf("Enter status (SUCCESS, FAILURE, TIMEOUT): ");
    scanf("%19s", input);

    if (strcmp(input, "SUCCESS") == 0)
        status = SUCCESS;
    else if (strcmp(input, "FAILURE") == 0)
        status = FAILURE;
    else if (strcmp(input, "TIMEOUT") == 0)
        status = TIMEOUT;
    else {
        printf("Invalid Status\n");
        return 0;
    }

    switch (status) {
        case SUCCESS:
            printf("Operation Successful\n");
            break;
        case FAILURE:
            printf("Operation Failed\n");
            break;
        case TIMEOUT:
            printf("Operation Timed Out\n");
            break;
    }
    return 0;
}
```

Explanation

The Status enumeration represents the three possible operation states. The program compares the user input with the status names and stores the matching enum value. The switch statement then prints the message associated with that status. Therefore, entering FAILURE prints Operation Failed.

Test Cases

Test Case	Input	Expected Output
1	SUCCESS	Operation Successful
2	FAILURE	Operation Failed
3	TIMEOUT	Operation Timed Out

Sample Output

```
Enter status (SUCCESS, FAILURE, TIMEOUT): FAILURE
Operation Failed
```

Result

Thus, the C program successfully uses enumeration to represent operation status and prints the corresponding message.